

Interview Preparation Guide — Rohit Jibhakate

Senior AI/ML Engineer | LLMOps • RAG • Agentic AI • GenAI  [linkedin.com/in/rohit-jibhakate](https://www.linkedin.com/in/rohit-jibhakate)

Compiled from real interview experiences across EY, Capgemini, EPAM, LTI, Andela, and Bajaj Finance.

Use this as a high-signal prep sheet — every topic here was asked in an actual round.

Table of Contents

1. [RAG \(Retrieval-Augmented Generation\)](#)
 2. [Agentic AI & LangGraph](#)
 3. [LangChain](#)
 4. [LLMOps & Monitoring](#)
 5. [Python — Core & Advanced](#)
 6. [Machine Learning](#)
 7. [SQL & Databases](#)
 8. [System Design](#)
 9. [Web / API Concepts](#)
 10. [Coding Problems](#)
-

RAG

Heavy focus area — EY Round 1 was entirely RAG-based. Expect scenario questions, not just theory.

Pipeline Stages

- **Injection:** docs → chunks → embeddings → Vector DB
- **Querying:** query → vectorizer → retrieve (top_k, semantic similarity) → re-rank → LLM
- **Generation:** prompt construction + LLM call + output

Chunking

- What chunking strategy do you use? (fixed-size, recursive, semantic)
- How do you handle **image and tabular data** during chunking? (multimodal RAG, table-aware splitting)

- How does **metadata tagging** work and why is it important for filtering?

Embeddings & Vector DB

- Which embedding model do you use and why?
- How do you handle **stale embeddings** when source data changes?
- How do you verify a policy/document is properly stored and indexed?
- How to confirm a specific chunk is retrievable via a test query?

Retrieval & Re-ranking

- How does **hybrid search** work? (dense + sparse / BM25 + semantic)
- Why add keyword search as a fallback?
- How do you find the **optimal top_k**?
- During re-ranking, how do you ensure a relevant chunk isn't dropped?
- What to do when most retrieved chunks are similar (redundancy)?

Debugging in Production

- Chunks retrieved are relevant but the **answer is wrong** — how do you debug?
- What if the **answer relevancy is poor** even when context and KB are correct?
- How to check for **retrieval failure** vs **hallucination**?
- How to validate that a freshly uploaded policy is being returned correctly?

Caching

- How does **response caching** work in RAG?
- What is **cache validation** and when do you invalidate?

Security

- How do you **prevent prompt injection** in a RAG pipeline?

Scaling

- How do you process **10M documents** efficiently?
- How do you **decrease latency from 1 min to 10 sec**?

RAGAS (Evaluation Framework)

- Metrics: Faithfulness, Answer Relevancy, Context Precision, Context Recall
- How do you use RAGAS in production evaluation?
- When would RAGAS flag a false positive/negative?

RAG vs Fine-tuning

- When to use RAG vs fine-tuning?
 - Trade-offs: cost, latency, accuracy, data privacy
-

Agentic AI & LangGraph

Asked in EY Round 2, Andela, and Capgemini. Focus on production robustness.

Agents & Tools

- How do **agents interact with each other** in LangGraph / LangChain?
- How does **tool calling** happen internally?
- How do you handle **tool failure** (retry, fallback, graceful degradation)?
- What is **exponential backoff** and when do you apply it?

State & Checkpointing

- How do you **design and manage state** in a LangGraph pipeline?
- When and where do you **place checkpoints** in your pipeline?
- How does **checkpointing ensure resumability** after failure?
- How to implement **safe parallel processing** in LangGraph?

Agent Failure & Hallucination

- How do you handle **agent failure** mid-pipeline?
- How do you **detect hallucinated answers** in production?
- After AI-generated code for a full project — how do you review it **beyond SME review**?
 - Static analysis, unit test generation, contract testing, LLM-as-a-Judge

Observability

- Can you use **Langsmith / Langfuse** for clients whose data cannot leave their environment?
 - Self-hosted Langfuse, on-prem deployments, data residency considerations

LangChain vs LangGraph

- Key architectural differences?
 - When would you choose one over the other?
-

LangChain

Capgemini L2 focused heavily here.

- **Runnable sequence vs traditional chain** — what changed in LangChain v0.2+?
 - What are **callbacks** in LangChain and how do you use them?
 - How do you **optimize token usage** in LangChain?
 - How does **chaining** work internally?
 - How do you handle **retry and fallback** in LangChain / LangGraph?
-

LLMOps & Monitoring

Recurring across EY, Andela, Capgemini.

- How do you **monitor AI systems in production**?
 - Metrics: latency, token usage, error rates, answer relevancy drift
 - Tools: Langfuse, LangSmith, Prometheus, Grafana
 - How do you **test AI-generated code** for production readiness?
 - What do you do when an **API call fails**?
 - Retry with backoff, circuit breaker, fallback model
 - How do you handle **hallucination detection** post-deployment?
-

Python — Core & Advanced

EPAM explicitly said "Do core Python." LTI and Capgemini also asked Python.

Concurrency

- **Multithreading vs Multiprocessing** — when to use which?
- What is the **GIL (Global Interpreter Lock)** and how does it affect concurrency?
- How do you **prevent race conditions** in multithreaded code?

OOP

- What is a **class** in Python?
- What are **class methods vs static methods** — why use static methods?
- What is **method overloading** and how does Python handle it?
- **MRO (Method Resolution Order)** and `super()` — how does Python resolve multiple inheritance?
- What are **metaclasses** and when would you use them?
- What are **descriptors**?

Functions & Closures

- **Decorators** — how do they work, write one from scratch
- **Generators vs Iterators** — difference, use cases, `yield`
- **Mutable default arguments** — what's the gotcha?

Memory & Performance

- **Deep copy vs shallow copy** — when does shallow copy cause bugs?
- How do you **optimize memory/storage** in Python?
- **Context managers** — `__enter__` / `__exit__`, using `contextlib`

APIs

- **FastAPI vs Flask** — when to use each?
- **ORM in FastAPI** — SQLAlchemy, async ORMs
- What are **middlewares**?

Keyword Arguments

- Difference between **positional and keyword arguments**
 - `*args` and `**kwargs` patterns
-

Machine Learning

Bajaj Finance L1 went deep into ML fundamentals. Capgemini and LTI also touched this.

Algorithms

- **K-Means Clustering** — how it works, choosing k, inputs required
- **Bagging vs Boosting** — Random Forest vs XGBoost/AdaBoost
- How would you train a model for **sales prediction** from historical data?

Handling Data

- How do you handle **imbalanced data**?
 - Oversampling (SMOTE), undersampling, class weights, threshold tuning
- **Regularization techniques** — L1 (Lasso), L2 (Ridge), Elastic Net — what they do and when to use

Evaluation Metrics

- **F1 Score** — harmonic mean of precision and recall, use when data is imbalanced

- Precision vs Recall tradeoff
- Why prioritize **recall** for certain use cases (e.g., adverse drug events, relevant news)

Fine-tuning (BERT example from Bajaj)

- Problem: Binary classification (relevant/not relevant)
 - Data: ~4,000 SME-labeled rows
 - Tokenization: BERT tokenizer, fixed-length, padding, attention masks
 - Training: Freeze base layers, fine-tune CLS head, binary cross-entropy, ~10-15 epochs
 - Validation: 10% holdout vs proper cross-validation (limitation to acknowledge)
 - Deployment: Flask API + Azure Functions
-

SQL & Databases

Asked in EY Round 2, LTI, Capgemini.

- **Primary key vs Foreign key**
 - All types of **JOINS** — INNER, LEFT, RIGHT, FULL OUTER, CROSS, SELF
 - **Query:** Given tables `employee`, `department`, `salary` — increase salary and show results for a specific department
 - **Primary/Foreign key** relationships in a multi-table schema
-

System Design

EY Round 1 and LTI asked system design.

- **Monolithic vs Microservices architecture** — trade-offs, when to migrate
 - How to **secure, scale, and optimize AI systems**?
 - How do you design for **10M document ingestion** in a RAG pipeline?
 - How to add **API failure resilience** — retries, circuit breakers, queuing
-

Web / API Concepts

EY Round 2 asked React; EPAM asked API/middleware concepts.

- **Virtual DOM** in React — what problem does it solve?
- **Props** in React — how data flows between components
- **Middlewares** — what they are and how they fit into a request lifecycle

- **GCP tools used via code** (not UI) — Cloud Run, Vertex AI, Firestore, Secret Manager, Artifact Registry
-

Coding Problems

Every interview had at least one coding question. Prepare these cold.

Problem	Company	Approach
Duplicate elements in a list (preserve order)	Bajaj	Dict for frequency count, append to result list
Monthly revenue by city (pandas)	Bajaj	<code>groupby(['city', 'month']), sum(price * qty)</code>
Run-length encoding (<code>(aaaabbccc → a4b2c3)</code>)	Capgemini	Iterate with a counter, flush on char change
Valid parentheses	EPAM	Stack — push open, pop on close
Triplet sum (3Sum)	EPAM	Sort + two pointers
Is <code>(s)</code> a subsequence of <code>(t)</code> ?	LTI	Two-pointer on both strings
Lambda & list comprehension (<code>(mult, x**2)</code>)	EY Round 2	Know lambda syntax, comprehension edge cases

💡 Key Takeaways from Rohit's Interview Experiences

1. **RAG is non-negotiable** — every company asked it. Know every stage deeply, especially debugging and evaluation.
2. **Core Python matters** — EPAM said it outright. Even senior roles test GIL, decorators, and generators.
3. **Agentic AI = production thinking** — interviewers want to know what happens when things fail, not just the happy path.
4. **Coding is always there** — at least one coding question per round, often a second one. Practice daily.
5. **ML fundamentals are back** — Bajaj went deep. Know k-means, bagging/boosting, F1, and regularization cold.
6. **Be ready to talk about your projects end-to-end** — every interviewer asked for a project deep dive.

Built from real interview notes by Rohit Jibhakate  *Follow for more: [linkedin.com/in/rohit-jibhakate](https://www.linkedin.com/in/rohit-jibhakate)*